Amortized Bayesian parameter recovery for dynamical systems via conditional flow matching

Technical report on the `amortix` package, v0.1

Ivan Oseledets

August 2026

Abstract

We consider the inverse problem of recovering the parameters of a dynamical system — an ODE, an SDE, or a PDE — from noisy observations of its trajectory at a finite set of time points, in the Bayesian setting: the answer is the posterior distribution over parameters, not a point estimate. The `amortix` package implements an *amortized* solver: a single neural network is trained once, on simulated (parameters, data) pairs, and thereafter produces thousands of posterior samples for any new dataset in a fraction of a second, with no likelihood evaluations and no per-dataset optimization. The generative engine is conditional flow matching: the network learns a velocity field that transports a standard Gaussian onto the posterior, conditioned on the data. A single trained model handles *any number and placement of observation points* — from a handful of irregular measurements to near-continuous monitoring — which is the setting required for sequential experiment refinement and optimal design of measurements. This report gives the problem setup and the method from scratch, describes the architecture in plain terms, defines the evaluation protocol, and summarizes benchmark results on fifteen systems, including a stochastic-volatility model with a hidden state, a jump-diffusion, the Hodgkin–Huxley neuron, a pharmacokinetic model, and a reaction–diffusion PDE. On the fixed-design benchmark, 31 of 32 parameters across nine systems pass strict calibration tests; on the variable-design zoo, posteriors agree with exact-likelihood references up to residuals that decay smoothly with training compute. A measured head-to-head against neural posterior estimation from the `sbi` package at equal simulation budget and default settings makes the case for the architecture: on raw prices NPE misses the volatility by $+0.45$ posterior-sd at $2.6\times$ the correct width — exactly the failure our conditioning analysis predicts — while this model is statistically indistinguishable from the exact posterior on the same data; we also report training and inference wall-clock for both, and for exact-likelihood MCMC. Finally we document, in plain language, the three failure modes we encountered — all traceable to the conditioning of the encoder's input — and their cures.

1 Problem setup

Let a dynamical system depend on an unknown parameter vector $m \in \mathbb{R}^d$ with a known prior $p(m)$ (in all our benchmarks, uniform on a box). The system produces a state trajectory $x(t)$ — through an ODE $\dot{x} = f(x; m)$, an SDE $dx = f(x; m)\,dt + \sigma(x; m)\,dW_t$, or a discretized PDE — and we observe it through a noisy measurement channel at K time points $t_1 < \cdots < t_K$:

$$y_i = h\big(x(t_i)\big) + \varepsilon_i, \qquad \mathcal{D} = \{(t_i, y_i)\}_{i=1}^{K}. \tag{1}$$

The observation operator h may hide part of the state (e.g. only the price is seen while the volatility is latent; only the voltage of a neuron is seen while the gating variables are not; only three spatial sensors of a PDE field are read). The goal is the posterior

$$p(m \mid \mathcal{D}) \;\propto\; p(\mathcal{D} \mid m)\, p(m). \tag{2}$$

1.1 What makes this hard

It is worth being explicit about the difficulties, because each one below defeated at least one plausible architecture during development, and the design choices of Secs. 3–4 are answers to them, not aesthetics.

1. **The likelihood is intractable or expensive.** For SDEs observed at gaps, $p(\mathcal{D} \mid m)$ integrates over the latent path between observations; for models with hidden states (stochastic volatility, gating variables of a neuron) even the observed-process transition density is unavailable. Classical estimators exist per problem, but each is a bespoke derivation.

2. **The answer is a distribution, and its *width* must be right.** A point estimate can be checked against the truth; a posterior claims a full uncertainty statement, and the failure modes are asymmetric: an overconfident posterior (too narrow) is silently wrong in the way that matters most for downstream decisions. Verifying width requires reference posteriors or calibration tests, and the standard calibration test has limited statistical power (Sec. 5) — so even *knowing* that you failed is nontrivial.

3. **One network must serve all designs.** Under design amortization the same weights answer $p(m \mid S)$ for $K = 5$ irregular points and for $K = 100$ dense ones. The posterior width is governed by K, yet the natural set architecture (normalized attention) is provably poor at communicating set size (Sec. 3); and the training distribution over designs becomes a modeling decision with measurable consequences (Sec. 4).

4. **Conditioning of the raw data.** Multiplicative processes float over an order of magnitude of scale; jump-diffusions produce squared-increment features with standard deviation 3×10^4 next to signals of order 10^{-2}; the useful coordinate (e.g. the logarithm) differs per problem and must be *found*, not prescribed. Every hard failure we encountered traced to this (Sec. 8).

5. **Weak identifiability.** Real posteriors have ridges: in the reaction–diffusion problem only the combination $2\sqrt{Dr}$ (the front speed) is sharply determined, and the flow's residual error concentrates precisely along the flat direction, where the training gradient is weakest (Sec. 6.3).

6. **Fast and repeated inference.** Calibration studies, sequential experiments, and design-of-experiments loops evaluate posteriors for thousands of datasets and candidate designs; a per-dataset MCMC or optimization in the loop is the bottleneck this method exists to remove (Sec. 7).

Amortization. Simulation-based (likelihood-free) inference addresses difficulties 1 and 6 directly (the rest are what the rest of this report is about). Sampling from the joint is easy — draw $m_j \sim p(m)$, simulate the system, apply the observation channel — and yields training pairs $(m_j, \mathcal{D}_j)$. A conditional generative model $q_\theta(m \mid \mathcal{D})$ trained on these pairs approximates the posterior *for every dataset simultaneously*: the cost of inference is moved into a one-time training phase (hence "amortized"). At deployment, posterior sampling is a forward pass.

Design amortization. We additionally require that a *single* trained network be correct for any number K and any placement of observation times (and, for PDEs, sensor positions). Formally, if S is a subset of the measurement points of one experiment, the network must approximate $p(m \mid S)$ coherently for *all* subsets S — e.g. the posterior must widen honestly when points are removed. This is precisely the object needed for optimal experimental design, where candidate designs are compared by the posteriors they would produce.

2 Method: posterior sampling by conditional flow matching

Parameter normalization. Since the prior is a product of known one-dimensional distributions, we map parameters componentwise through their prior CDF and the standard normal quantile,

$$z = \Phi^{-1}\big(F_{\text{prior}}(m)\big) \in \mathbb{R}^d, \tag{3}$$

so that in z-coordinates the *prior* is exactly $\mathcal{N}(0, I)$ and box constraints disappear. This choice has a practical consequence: when the data carry no information, the posterior equals the prior equals the base distribution, and the model has nothing to learn — the uninformative limit is the zero of the parametrization, not a nontrivial target.

Flow matching. We represent the posterior as the endpoint of a learned transport. Let $z_0 \sim \mathcal{N}(0, I)$ be base noise and z_1 the (normalized) parameter drawn jointly with its dataset $\mathcal{D}$. Connect them by the straight line

$$z_t = (1 - t)\, z_0 + t\, z_1, \qquad t \in [0, 1], \tag{4}$$

and train a velocity network v_θ to regress the direction of this line, conditioned on the data:

$$\mathcal{L}(\theta) \;=\; \mathbb{E}_{m, \mathcal{D}\sim p(m)p(\mathcal{D}|m),\; z_0\sim\mathcal{N}(0,I),\; t\sim U[0,1]} \left\| v_\theta(z_t,\, t,\, \mathcal{D}) - (z_1 - z_0) \right\|^2. \tag{5}$$

By the conditional flow matching theorem [1], the minimizer of (5) is the *marginal* velocity field whose flow transports $\mathcal{N}(0, I)$ at $t = 0$ onto the conditional distribution $p(z_1 \mid \mathcal{D})$ at $t = 1$ — i.e. onto the posterior in z-coordinates. Intuitively: the network sees a cloud of straight lines from noise to posterior samples and learns their average direction at every $(z, t, \mathcal{D})$; following that averaged field deforms the Gaussian cloud into the posterior cloud.

Sampling. Given a new dataset $\mathcal{D}$, draw $z_0 \sim \mathcal{N}(0, I)$, integrate $\dot{z} = v_\theta(z, t, \mathcal{D})$ from $t = 0$ to 1 (we use 20 midpoint steps; finer solvers were tested and change nothing, Sec. 6.3), and map back through the inverse of (3). Repeating with fresh z_0 gives i.i.d. posterior samples. Evaluation uses an exponential moving average of the training weights.

Note what is *not* required: no likelihood, no adjoint of the simulator, no per-dataset MCMC or optimization. The simulator is only ever run forward, during training-set generation.

3 Architecture

The network has two parts: an *encoder* that turns the observation set $\mathcal{D}$ into context vectors, and the *velocity network* v_θ that consumes them. The design brief for the encoder was strict universality: no hand-crafted, problem-specific features; every transformation of the raw data must be learnable and shared across all problems. The default model is small — about 5×10^5 weights (width 64, three encoder blocks and three velocity blocks) — and Sec. 6.3 shows capacity is not the accuracy bottleneck.

3.1 From observations to tokens

Each observation becomes a token (a feature vector). Two constructions are used, chosen by an explicit and empirically verified rule.

Learnable axis reparametrization. Both constructions share one module: a monotone map $w(\cdot)$ applied to observed values (and to time gaps), parametrized as a positive mixture over a bank of signed-logarithmic scales $\{\, \mathrm{sign}(x) \log(1 + |x|/s_k) \,\}$ with learnable weights. The mixture spans the identity and the logarithm continuously, so the network can *learn* the coordinate in which the process is well-conditioned (e.g. log-price for geometric Brownian motion) instead of having it prescribed. A cautionary measurement guided this design: a more general Yeo–Johnson transform also contains the logarithm in its parameter space (at $\lambda = 0$), yet gradient descent never found it — membership in the hypothesis class is not the same as reachability from the initialization. The mixture parametrization keeps the useful coordinates inside the basin that gradients actually explore.

Point tokens (the general case). Each observation contributes the token

$$\big[\, t_i,\; w(y_i),\; \text{channel/sensor id},\; \text{set-size feature} \,\big].$$

Nothing else is assumed about the process.

Pair tokens (when the observed process is Markov). If the *observed* coordinate is itself Markov — as for scalar SDEs observed directly — the likelihood factorizes over consecutive observations, $p(\mathcal{D} \mid m) = \prod_i p\big(y_{i+1} \mid y_i; m\big)$, and the sufficient arguments of each factor are the pair (y_i, y_{i+1}) and the gap Δt_i. We exploit this by building one token per consecutive pair (after sorting by time): $\big[\, t_i,\; w(y_i),\; \Delta w_i,\; (\Delta w_i)^2,\; w_t(\Delta t_i) \,\big]$, where $\Delta w_i = w(y_{i+1}) - w(y_i)$. These are exactly the inputs a transition density needs, presented in a learnable coordinate — an inductive bias, not a hand-crafted statistic. Two additions make this construction robust: squared increments are logarithmically compressed before

normalization (heavy-tailed processes such as jump-diffusions otherwise destroy the feature scaling — Sec. 8), and a set-conditioning block (statistics pooled over the whole token set modulate each feature channel, with the modulation initialized to the identity) lets the normalization adapt to each dataset.

The class rule. Pair tokens are used *iff* the observed process is Markov; point tokens otherwise. The rule was verified in both directions: a pair-token model trained on one Markov system transfers cleanly to another (geometric Brownian motion → Ornstein–Uhlenbeck), while on a stochastic-volatility model, where the observed price is *not* Markov (the volatility is hidden), the pair construction loses its advantage entirely — point and pair models become statistically indistinguishable, as the factorization argument predicts. The package selects the construction automatically from a per-problem Markovianity flag.

3.2 Set encoding: time, permutation invariance, and set size

The tokens are processed by a transformer encoder. Three details matter and were each isolated by experiment.

Time enters through attention phases. Standard rotary position embeddings encode the *index* of a token, which is meaningless for irregularly sampled data — and worse than meaningless: we measured that index-based encoding creates a hidden train/test contract on token placement, producing systematic biases of 1–3 posterior standard deviations with no code being "wrong". Instead, the attention phases are computed from *physical time* t_i through a geometric bank of frequencies. The result is exact permutation invariance of the encoder (verified to 10^{-8}): the model sees a *set* of time-stamped measurements, not a sequence.

Normalized attention cannot count. Posterior *width* is governed by the number of observations K (in regular problems, error $\sim K^{-1/2}$), but softmax attention computes weighted *means* over the set — and means erase cardinality. We measured this directly: linear probes on the encoder's pooled output recover the posterior-mean statistics with $R^2 = 0.83$–0.94 in every architecture variant, while $R^2(\log K)$ collapses exactly in the variants whose posterior widths fail. The cure is one bit of honesty, not process knowledge: append $\log K$ as a feature (or let two learned "register" tokens accumulate the count). With it, dense-design widths drop from ratio ≈ 1.95 to 1.49 at $K = 100$ without touching bias.

Velocity network. The current flow state z_t is processed by self-attention over the d parameter slots; the flow time t and the pooled data context modulate every block through adaptive layer norms whose modulation is initialized to zero (adaLN-Zero, [5]) — training starts from an identity-like map and departs only as the data warrant.

4 Training protocol for design amortization

Training data are simulated on a fine grid once; observation designs are then *resampled from the stored fine paths at every optimizer step*. The recipe has three components, each fixed by a controlled comparison (systems: geometric Brownian motion, Ornstein–Uhlenbeck, FitzHugh–Nagumo):

1. **Fresh designs every step.** Each trajectory's observation subset is re-drawn at every optimizer step. If instead each trajectory keeps one fixed design, a long training run fits the dense designs but *overfits the design set*: at sparse K the model becomes overconfident (posterior width ratio down to 0.86 against the exact reference). Fresh resampling makes design memorization impossible, and one simulation serves unboundedly many designs — which is also exactly the training mode that makes $p(m \mid S)$ coherent across subsets S, as required for experiment design.
2. **The size distribution of designs.** K is drawn 50% log-uniform on $[K_{\min}, K_{\max}]$ and 50% uniform on the dense half of the range. Log-uniform alone starves the dense regime (only $\sim 6\%$ of mass lands at $K \geq 100$), which showed up as inflated widths precisely there; the mixture alone halves the dense-design excess (e.g. width ratio $1.27 \to 1.14$ on geometric Brownian motion).
3. **Budget scaling.** When more accuracy is needed, the number of simulations and optimizer steps are scaled *together* (constant ≈ 38 epochs). Section 6.3 shows the residuals then decay as clean power laws.

With this recipe, on geometric Brownian motion the posterior width matches the exact per-design posterior to within 5–7% uniformly over $K = 5, 9, 20, 100$.

5 Evaluation protocol

Two instruments are used, with distinct roles.

Simulation-based calibration (the screen). SBC [2] checks a necessary condition of Bayesian correctness: if $m^* \sim p(m)$, $\mathcal{D} \sim p(\mathcal{D} \mid m^*)$, then the rank of m^* among posterior samples must be uniform; departures are tested per parameter (we report the p-value of a uniformity test; $p > 0.05$ passes). SBC needs no reference posterior and runs on every system, but its statistical power at practical sizes is limited, and we measured the limit: at 300–500 simulated datasets it is blind to posterior-width errors up to $\approx 30\%$ and its p-values flicker between retrains when residual biases are 0.1–0.2 posterior-sd. It also cannot penalize a posterior that simply returns the prior. SBC is therefore used as a cheap screen, never as the final verdict.

Reference posteriors (the record). Wherever a tractable likelihood exists we compare *distributions* on the same observed points: the signed error of the posterior mean in units of the reference posterior's standard deviation ("bias"), and the ratio of standard deviations ("width", 1 = perfect, > 1 = conservative, < 1 = overconfident). References are closed-form where available (geometric Brownian motion, linear-Gaussian) and otherwise adaptive Metropolis MCMC with exact likelihoods (a Poisson-mixture transition density for the jump-diffusion; log-normal residuals around the pharmacokinetic curve; Gaussian noise around the PDE solve). Chains are validated by multi-start agreement before being trusted; where both a chain and a closed form exist, the chains reproduce the truth to worst mean discrepancy 0.099 posterior-sd and widths 0.91–1.03 over 24 comparisons.

Twice during development the two instruments contradicted each other, and both times the contradiction exposed a real bug (a padding mask leaking into set-level statistics; the index-based positional contract of Sec. 3). Two disagreeing instruments are worth more than either alone.

Encoder forensics. A third diagnostic separates *extraction* failures from *downstream* failures: linear (ridge) probes from the encoder's pooled context onto the problem's known sufficient statistics. In the jump-diffusion autopsy the failing model encoded the robust statistics *better* than the healthy one ($R^2 = 0.967$ on truncated realized variance) — the information was extracted, and the defect was in the representation's stability, not its content. This redirected the fix (Sec. 8).

6 Benchmark results

6.1 Fixed-design gallery: nine systems

Nine systems with a shared observation grid per problem: linear-Gaussian (sanity case with an exact posterior), Ornstein–Uhlenbeck, geometric Brownian motion, Cox–Ingersoll–Ross, a double-well SDE, stochastic Lotka–Volterra, FitzHugh–Nagumo, SEIR, and an SDE with a cubic drift to be discovered (SINDy-style). Across the 32 parameters, 31 pass SBC; the single miss sits at the threshold ($p = 0.04$). Mean calibration error is 1.3 percentage points. On the cases with exact references, the learned posteriors extract 89–90% of the available information (width vs. the exact posterior).

6.2 Variable-design zoo: six systems

All trained with the recipe of Sec. 4, evaluated by the protocol of Sec. 5 over mixed random designs plus fixed-K buckets.

system	d	tokens	outcome
Heston stochastic volatility	5	point	all 25 SBC cells pass; pair tokens give no advantage, confirming the class rule (price is not Markov)
Merton jump-diffusion	5	pair	25/25 after the heavy-tail cure of Sec. 8; residual width on the budget curve below
Hénon–Heiles oscillator [6]	3	point	calibrated at every K down to 6; sparse designs yield honestly wide posteriors, not miscalibration
Hodgkin–Huxley neuron	4	point	calibrated on mixed designs; one dense-design cell shows the budget signature
Pharmacokinetics (Bateman)	3	point	at $K = 6$ blood draws: bias -0.01 to -0.06 sd, widths 1.09–1.33 vs. exact-likelihood MCMC — calibrated clinical-regime inference in a fraction of a second
Fisher–KPP reaction–diffusion PDE	2	point	posterior concentrates on the ridge $D \cdot r \approx$ const; the residual along-ridge bias decays with budget (below)

6.3 Residuals decay as power laws in training compute

The two stubborn residuals of the zoo were driven down by scaling training budget (simulations and steps together, Sec. 4) at *fixed* model size:

	$\times 1$ budget	$\times 3$	$\times 6$
jump-diffusion: width excess, hardest cell	0.246	0.120	0.059
reaction–diffusion: along-ridge bias (sd)	$+0.41$	$+0.23$	$+0.14$
reaction–diffusion: front-speed bias (sd)	-0.21	-0.14	-0.075

The width excess decays $\propto 1/\text{budget}$; the ridge bias $\sim \text{budget}^{-1/2}$ — the weakly identified direction of the posterior (weakest gradient signal in the loss (5)) converges last, with no wall. Two negative controls complete the picture: doubling network width and depth at fixed budget changes almost nothing (1.103 vs. 1.120), and refining the sampling ODE solver (RK4 with 60 steps vs. midpoint with 20) changes nothing at all. Remaining inaccuracy is a compute price list, not an open problem.

7 Computational cost, and a measured head-to-head with `sbi`

Claims about speed and about other packages should be measured, not asserted. We ran the directly comparable amortized method — neural posterior estimation (NPE) as implemented in the `sbi` package (v0.27), the reference implementation used by the community benchmark [3] — on the one problem where the ground truth is beyond argument: geometric Brownian motion with the package's fixed design (95 observed grid values), scored against the exact closed-form posterior. Both learners received the *same simulation budget* (20 000 draws from the same prior and simulator), both ran with their default hyperparameters and their own stopping rules, on the same CPU (Apple M-series laptop). Accuracy is measured on 200 test datasets: signed bias of the posterior mean in units of the exact posterior's standard deviation, and the width ratio (method sd / exact sd). Inference draws 2 000 posterior samples per dataset. The script is `examples/baseline_npe.py`; per-arm JSON is under `results/`.

method (input given to it)	train	inference	μ: bias / width	σ: bias / width
`amortix` CFM, raw prices (defaults)	488 s	426 ms	$+0.00$ / 1.00	-0.00 / 1.05
`sbi` NPE, raw prices (defaults)	29 s	13 ms	-0.02 / 1.06	$+0.45 \pm 0.19$ / 2.64
`sbi` NPE, hand log-prices	34 s	12 ms	-0.04 / 1.01	$+0.32 \pm 0.17$ / 2.29
`sbi` NPE, hand log-returns	10 s	13 ms	$+0.07$ / 1.07	$+0.13 \pm 0.10$ / 1.44
exact-likelihood MCMC	—	42 ms	exact up to chain error (bias -0.04 / $+0.01$)	

Three readings of this table, in decreasing order of importance.

Accuracy at defaults. `amortix` is statistically indistinguishable from the exact posterior on both parameters. Default NPE misses the volatility badly: +0.45 sd bias at 2.6× the correct width. The mechanism is precisely guise 1 of Sec. 8 — raw prices have a floating multiplicative scale, and z-scoring (NPE's default preprocessing) cannot fix a per-dataset scale; `sbi` itself prints a warning about "extreme outliers … precision loss during z-scoring" on this data. This is not a defect of NPE as a method; it is what happens to *any* conditional density estimator whose input representation is left to defaults on multiplicative data.

The hand-engineering ladder. Giving NPE progressively more of the analytically known answer — log-prices, then per-gap log-returns (the exact coordinates of the likelihood factorization) — improves it monotonically, yet even the log-return arm leaves the volatility 44% too wide at this budget: the conditioner must still assemble the quadratic variation internally. With fully hand-derived sufficient statistics NPE would presumably close the gap — but that regime requires having already solved the inference problem analytically, which is exactly the dependence this package exists to remove. `amortix` reaches the exact posterior *from raw prices*, because the learnable axis map and the increment-based pair tokens of Sec. 3 discover those coordinates during training.

Cost, honestly. At these defaults NPE trains $\sim 17\times$ faster and samples $\sim 30\times$ faster than `amortix`, and on this particular problem — a tractable likelihood, a single dataset — plain MCMC costs 42 ms and is exact: nothing amortized is worth it there. But read the MCMC row for what it is: it exists only because GBM's likelihood is a three-line Gaussian formula, which is precisely why GBM was chosen to referee the two *learned* methods. Transplanted to the rest of the zoo, the row changes character. For the reaction–diffusion problem one likelihood evaluation is one PDE solve (measured: 16 ms), so one MCMC dataset costs ≈ 3 minutes of PDE solving against the network's half-second — a $\sim 400\times$ gap that multiplies over every dataset and every candidate design. And for the stochastic-volatility and neuron models the row does not exist at all: there is no likelihood formula to evaluate, at any price. The amortized method earns its training bill when (a) no tractable likelihood exists (the stochastic-volatility, neuron, and most zoo cases), (b) thousands of datasets or candidate designs must be processed (calibration studies and design-of-experiments loops; `amortix` encodes and integrates whole batches jointly), and (c) the input representation must not be a per-problem decision — the first reading above priced that decision at a +0.45 sd bias. For scale on modern hardware: one variable-design zoo case at base budget (20 000 simulations, 12 000 steps, the $\approx 5 \times 10^5$-parameter model) trains in 8–12 minutes on a single H200 GPU; the $\times 6$-budget runs behind the sharpest numbers of Sec. 6.3 take about 50 minutes; the fixed-design gallery cases train in 8–45 minutes on a laptop CPU.

8 Failure anatomy: one disease, three guises

Every hard failure we encountered across fifteen systems reduced to the same cause: the *conditioning of the encoder's input*. It appeared in three guises, and each cure is a learnable, problem-agnostic module:

guise	symptom	cure
floating multiplicative scale	+0.48 sd bias on volatility	learnable monotone axis map (Sec. 3)
set-size blindness	widths 1.5–2.5× at dense designs	explicit $\log K$ / register tokens
heavy tails (jumps)	squared increments, sd 3×10^4	log-compression + set conditioning

The third guise deserves one paragraph, because its autopsy illustrates the method of the whole project. On the jump-diffusion, the pair-token model failed with a +1.12 sd bias. Probing (Sec. 5) showed the relevant statistics were extracted *better* than in healthy models — the failure was that jump outliers set the running normalization scale (standard deviation 3×10^4 against a diffusion signal of order 10^{-2}–1), so the flow trained against a drifting representation. Log-compressing the increment features (near-identity on clean diffusions, since $\log(1+x) \approx x$ for small x) fixed all 25 calibration cells. A precision probe then found a smaller residual (+0.27 sd) with its own mechanism: the effective jump/no-jump classification threshold floats with the unknown volatility. Here we raced a hand-crafted fix (per-dataset median rescaling) against a universal learned one (set-pooled statistics modulating feature channels, initialized to identity): $+0.094 \pm 0.054$ vs. $+0.127 \pm 0.055$ — statistically indistinguishable. This was the third such duel of the project, and the third draw. The emerging pattern: *the hand-crafted fix identifies the mechanism and the right initialization basin; the learnable module placed in that basin is what ships.*

9 Baselines: what any result here is compared against

Every claim is read against a ladder of comparisons (shipped in the package; details in `BASELINES.md`):

- **Controls.** Predicting the prior mean (exactly 25% mean absolute error on a uniform prior) and a degree-2 ridge regression on ~ 20 generic summary statistics — the cheapest serious estimator. These also unmask a failure mode SBC cannot see: a posterior that returns the prior passes SBC and fails both controls.
- **Classical per-case estimators.** Each benchmark problem ships the estimator a domain practitioner would use: exact maximum likelihood (Ornstein–Uhlenbeck, geometric Brownian motion), Euler pseudo-likelihood (Cox–Ingersoll–Ross), Kramers–Moyal / SINDy regression (drift discovery), multi-start nonlinear least squares (FitzHugh–Nagumo, SEIR, Lotka–Volterra), the exact posterior mean (linear-Gaussian).
- **Reference posteriors.** Closed forms and validated exact-likelihood MCMC (Sec. 5) — the instrument of record.
- **External packages.** BayesFlow [4], the `sbi` package, and the `sbibm` benchmark suite [3], with which our Hodgkin–Huxley and ecology/epidemiology cases deliberately overlap; the measured head-to-head with `sbi`'s NPE is Sec. 7 and ships as `examples/baseline_npe.py`. The differentiators of `amortix` are the SDE-first problem zoo with real numerical solvers, single-model variable-design amortization, and the exact-reference evaluation harness.

10 Limitations and outlook

(i) Residual widths and ridge biases lie on measured power-law budget curves; sharpening them is a compute purchase, not a research question. (ii) The power limits of SBC mean that small residuals can only be certified where an exact or validated reference exists; systems without tractable likelihoods inherit the weaker screen. (iii) Optimal experimental design is the natural next step: the training mode of Sec. 4 already makes $p(m \mid S)$ coherent across subsets S of one experiment, which is the required primitive. (iv) Real-data studies with posterior predictive checks are not yet included.

A Mapping between this report and the package

For reproducibility, the plain-language names used above correspond to the following options of the `FlowPosterior` class (all are selected automatically by default):

in this report	in the package
point tokens	`embed="wpoint"`
pair tokens + set-conditioned modulation	`embed="wfilm"`
warped values/increments (fixed designs)	`embed="wbasis"` / `"wdiff"`
attention phases from physical time	`rope="time"`
fresh designs each optimizer step	`fit(retokenize=...)` via `make_retokenizer()`
variable-design problem wrapper	`amortix.designs.DesignProblem`
benchmark systems	`amortix.problems` (`DESIGN_ZOO` for Sec. 6.2)
calibration screen / reference probes	`sbc_design` / `amortix.mcmc` + likelihood factories

References

[1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. Flow matching for generative modeling. ICLR 2023.

[2] S. Talts, M. Betancourt, D. Simpson, A. Vehtari, A. Gelman. Validating Bayesian inference algorithms with simulation-based calibration. arXiv:1804.06788.

[3] J.-M. Lueckmann, J. Boelts, D. Greenberg, P. Gonçalves, J. Macke. Benchmarking simulation-based inference. AISTATS 2021.

[4] S. Radev, U. Mertens, A. Voss, L. Ardizzone, U. Köthe. BayesFlow: Learning complex stochastic models with invertible neural networks. IEEE TNNLS 2020.

[5] W. Peebles, S. Xie. Scalable diffusion models with transformers. ICCV 2023.

[6] C. Lubich, I. Oseledets, B. Vandereycken. Time integration of tensor trains. SIAM J. Numer. Anal. 53(2), 2015.